Table 12.2 Log levels

Log level	Description
`Trace`	The lowest level of logging, which results in all logs being output. This level is rarely used and should not be enabled in a production application, as it may contain sensitive information.
`Debug`	This level is intended to be used for writing messages that are useful while developing the application. For example, you might want to log the value of variables or the processing time of an operation. You are unlikely to log at this level in a live application.
`Information`	The information level is used for logging normal operations within the application. For example, you might want to log details of changes made to a `Property` record in the database, such as who changed the details and what the changes were.
`Warning`	Warning logs are used to record details of events that are unexpected or abnormal but don't raise an exception. For example, within the `CityModel OnGet` method, you might want to log a warning when route values don't match an existing record.
`Error`	This level is used for recording details of errors that stop the current request from processing—typically unhandled exceptions. The `ExceptionHandlerMiddleware` automatically creates a log when an exception occurs.
`Critical`	This level is used for recording details of catastrophic events that prevent normal operation of the application for all users, such as the server running out of disk space.

The `ILogger` interface is the primary abstraction for logging within .NET. It has a number of extension methods that determine the level of the log generated. They are nice and predictable in that they take the form `Log[Level]`, so to generate an informational log, you use the `LogInformation` method.

The next code examples show how to generate logs based on the name of the city passed in to the City page. If it is found in the database, you log an informational message. If not, you log a warning. First, you need to get a